

A WebAssembly Port of Grand Theft Auto III, Implemented in Java

UCS503

Submitted to: Professor Nisha Thakur

Thapar Institute of Engineering and Technology

Antriksh (1024031034)

Divyam Jain (1024031032)

1 Higher-order goal

This project aims to contribute to the preservation and cross-platform accessibility of a specific piece of legacy software by porting the Grand Theft Auto III engine to the browser via WebAssembly (WASM), so it can run directly inside a standard web browser without any native installation step. The engine is reimplemented natively in Java, informed by publicly documented reverse-engineering research (e.g., the work behind the *re3* project), with only a thin HTML shell around it.

2 Time-to-value

- We will deliver meaningful increments quickly by getting a minimal engine subset to boot in-browser within a short iteration window, before attempting full feature parity.
- We will prioritize fast feedback over perfection by using weekly iterations and CI-backed Java/TeaVM builds to reduce release friction.
- Success is defined by what can be run and measured in the browser in the first iteration (boot to an interactive state), then improved based on profiling and playtesting.

3 Problem Statement

Grand Theft Auto III, like most software of its era, was written against platform APIs such as native windowing, OpenGL/Direct3D, local-disk file I/O, and native threading. None of these have a direct browser equivalent, so the engine cannot run in a web browser without a porting layer.

We implement the engine from scratch in Java, informed by publicly documented reverse-engineering research into the original engine's file formats and game logic (e.g., the documentation produced by the *re3* project), rather than compiling or copying any existing C++ source.

Bringing the engine to the browser requires closing three technology gaps:

- **Rendering:** there is no native OpenGL/Direct3D in a browser; rendering code must target WebGL from Java.
- **Audio:** native audio backends have no browser equivalent; audio must be driven through the Web Audio API from Java.
- **Filesystem and concurrency:** the original engine assumes local-disk access and native threads, while browsers sandbox both behind browser-native storage and a restricted threading model.

Why this matters now (contextual relevance): a working browser build removes installation friction entirely, makes the engine trivially shareable for study, demonstration, or preservation,

and is a concrete, well-scoped systems-porting exercise that exercises real toolchain, performance, and platform-compatibility engineering.

4 Proposed Solution

4.1 Overview

Build a browser-playable WebAssembly build of the GTA III engine with the following components:

- **Language:** the engine and all game logic are implemented natively in Java; the browser side is a single, static HTML file plus a small amount of CSS.
- **Build pipeline:** TeaVM compiles the Java bytecode to a `.wasm` module plus a thin JS loader, so the same Java source also runs as an ordinary JVM build for local testing.
- **Rendering:** TeaVM's JSO interop module provides typed Java bindings to the WebGL2 API; rendering code is ordinary Java calling those bindings.
- **Audio:** JSO bindings to the Web Audio API, called from Java.
- **Input and asset loading:** JSO bindings to DOM keyboard/mouse/gamepad events and the browser's File API/IndexedDB, all invoked from Java – see §10 on copyright constraints for how assets are supplied.

4.2 Core workflow

1. The HTML page loads the TeaVM-generated WASM module and its small JS loader, then hands control to the Java entry point.
2. The user is prompted to supply local data files (from their own legally owned copy of GTA III); files are read via the File API and cached in IndexedDB, both called from Java through JSO, so future loads skip re-upload.
3. The engine's main loop runs inside the browser as a Java method bound to `requestAnimationFrame` via JSO.
4. Rendering, audio, input, and save/load proceed through the Java-side bindings described above.

4.3 Operational constraints for an educational, tier-2/3-hardware-friendly setting

- Target acceptable performance on mid-range consumer hardware in current Chrome and Firefox.
- Avoid dependence on novel rendering research; focus on toolchain correctness, Java-to-browser interop, and measured performance.
- Design for deployability on plain static web hosting (e.g. GitHub Pages) – no server-side compute or asset hosting required at runtime.

5 Solution Approach

This is an engineering porting project: the objective is a working, performant WASM build of the GTA III engine, implemented in Java, not new rendering or AI research.

5.1 Engine implementation

- Implement the engine’s core systems (world state, entity logic, a lightweight renderer) as plain Java classes, organized into a small number of packages.
- Keep the code within TeaVM’s supported JDK subset from the start (TeaVM has limited support for reflection and some standard-library classes), verified early via CI rather than discovered late.
- Isolate all browser-specific calls behind a thin JSO-binding layer, so the engine logic itself stays ordinary, testable Java.

5.2 Web architecture

- **Frontend:** a single static `index.html` with a canvas element, a file-input for local data assets, and a small settings panel (resolution, audio volume).
- **Build system:** Maven or Gradle with the TeaVM compiler plugin, producing `.wasm/.js` artifacts; no server-side runtime component is required.
- **Storage:** browser IndexedDB, accessed from Java via JSO, caches user-supplied data assets across sessions.

5.3 CI/CD and fast delivery enabler

- CI builds the TeaVM target on every commit and runs a headless smoke test (module loads, reaches an initialized render state).
- CD publishes build artifacts to a static-hosting staging environment for manual playtesting.
- Automated checks track WASM output size, boot time, and a scripted FPS benchmark.

6 Evaluation Criterion

6.1 Primary evaluation metric

Boot success and in-browser frame rate: whether the engine reaches an interactive state, and the frame rate sustained once there.

- Target: reach an interactive state in under 15 seconds after assets are loaded.
- Target: median ≥ 30 FPS at 720p during a standard test scene, in the pilot browser targets.
- Attribution: measured via an in-app frame-time counter logged during test sessions, and a boot-time timestamp from module init to first interactive frame.

6.2 Secondary evaluation metrics

- **Load time:** elapsed time from asset upload/cache-hit to playable state.
- **Output size:** compiled `.wasm` size relative to a plain JVM build of the same engine code.
- **Feature parity:** a checklist of rendering, audio, save/load, and input features verified against the documented original behavior.
- **Cross-browser compatibility:** a pass/fail matrix across Chrome, Firefox, and Safari.

6.3 Pilot validation plan

- Internal playtesting with 3–5 users who each supply assets from their own legally owned copy of GTA III.
- Compare FPS and load-time metrics against a plain-JVM build of the same engine on matched hardware as a baseline.

7 Scalability

1. **Scalable (theoretically):** keep the rendering, audio, input, and storage bindings modular and decoupled from game logic, so the engine can absorb a larger asset set or higher-resolution rendering targets without a structural rewrite.
2. **Operationally deployable (practically):** static-hosting-only deployment, versioned and reproducible build artifacts, and a CI/CD pipeline that regenerates a fresh WASM build on every change.

8 Engine availability heuristic

A mature set of tooling is available to implement this project entirely in Java, with a thin HTML shell:

- TeaVM, for compiling Java bytecode to WebAssembly (with a JS fallback path).
- TeaVM's JSO module, for typed Java bindings to the DOM, WebGL2, Web Audio, and browser storage APIs.
- Maven or Gradle with the TeaVM plugin, for build automation.
- JUnit, for engine-logic unit tests that run on the plain JVM, ahead of and independent from the browser build.
- Publicly documented reverse-engineering research on GTA III's file formats and engine behavior, used as a specification to implement against in Java.
- Browser developer tools, for WASM debugging and profiling.

We will use these mature components to focus on the Java engine, the browser bindings, and measurement, rather than inventing new infrastructure.

9 Project Scope and Deliverables

9.1 Initial deliverable

- TeaVM build pipeline compiling a minimal Java engine subset to WASM.
- Canvas boots, WebGL2 context initializes via JSO, and a static test scene renders.
- Basic input passthrough (keyboard/gamepad) verified end-to-end.
- CI: automated TeaVM build plus a headless boot smoke test on every push.

9.2 Subsequent deliverables

- Full game loop (main menu → gameplay) running in-browser.
- A clean separation between the JSO binding layer (rendering, audio, storage, input) and game logic, for maintainability.
- Local asset-loading UI with IndexedDB persistence.
- Save/load support using browser storage, and a performance tuning pass against the FPS/memory budget defined in §6.

10 Risks and Mitigations

- **Copyright and legal use:** GTA III's original code and data assets remain proprietary to Take-Two/Rockstar. *Mitigation* – our Java engine code is an original implementation written against publicly documented reverse-engineering research, not compiled or copied from any existing source; no copyrighted assets or original binaries are bundled, hosted, or redistributed. End users must supply data files from their own legally purchased copy of the game, loaded locally in their own browser session.
- **Reliance on third-party research:** the reverse-engineering documentation this project's specifications are based on (e.g. the research behind *re3*) has itself previously been the subject of a DMCA takedown against that project's source code. *Mitigation* – rely only on publicly available documentation of file formats and behavior, keep our own Java implementation and repository independent of that project's source, and be prepared to revisit specific details if a rights holder objects.
- **TeaVM/JDK compatibility:** TeaVM does not support the full JDK (notably, limited reflection and no true browser-side multithreading). *Mitigation* – restrict engine code to TeaVM's supported subset from day one, checked by CI, and design the main loop as single-threaded with cooperative scheduling.
- **Performance overhead:** Java-to-WASM via TeaVM can carry more overhead than hand-written C/Rust. *Mitigation* – profile early, keep hot loops (rendering, physics-lite updates) allocation-light, and avoid unnecessary object indirection in per-frame code.
- **Browser storage limits:** large asset sets may exceed IndexedDB quotas on some browsers. *Mitigation* – support streaming/partial loads and surface clear quota warnings to the user.

11 Summary

This project proposes an original Java reimplementation of the Grand Theft Auto III engine, informed by public reverse-engineering research, compiled to WebAssembly via TeaVM, so it can run directly in a browser without native installation. Implementation is entirely in Java, with only a thin HTML shell. It follows the course’s emphasis on time-to-value through an early bootable increment, CI/CD-backed iterative delivery, and measurable evaluation criteria (boot success, frame rate, load time). It is scoped as an engineering porting effort: no original game assets are redistributed by the project, and rights-holder copyright is respected throughout – end users provide their own legally owned data files locally.